VEEVA QA • PHASE 1

Formal Definitions & Quick Revision

Phase 1 • Textbook-style glossary, classifications and interview distinctions

- *Designed for quick revision and accurate interview answers.*
- *Definitions are carefully paraphrased from established software-testing terminology.*
- *Use this after the full guide; it is a revision companion, not a replacement for practice.*

Study method: understand the concept → explain it in your own words → apply it to a requirement → practise interview distinctions.

How to revise

For each term: read the definition, cover it, say it in your own words, then give an example. Do not memorize an example as if it were the definition.

A. Core concepts

Term	Formal revision definition
Software testing	A set of activities performed to evaluate software quality, determine whether requirements are satisfied, detect defects and provide information about risk.
Quality Assurance (QA)	Process-oriented activities intended to provide confidence that appropriate processes are established and followed to achieve quality.
Quality Control (QC)	Product-oriented activities used to evaluate whether a work product meets specified quality requirements.
Verification	Evaluation of a work product to determine whether it satisfies specified requirements: “Are we building the product correctly?”
Validation	Evaluation of a system to determine whether it meets user needs and intended use: “Are we building the right product?”
Requirement	A documented need, capability, condition or constraint that a product or system is expected to satisfy.
Test scenario / test condition	A high-level behavior, situation or condition to be tested.
Test case	A set of preconditions, inputs, steps and expected results designed to verify a test objective or condition.
Test data	Data used as input or setup when executing a test.
Expected result	The behavior or outcome defined by the requirement, specification or other accepted test oracle.
Actual result	The behavior or outcome observed during test execution.
Test execution	Running a test, comparing actual behavior with expected behavior and recording results/evidence.
Defect	A flaw in a work product or implementation that can cause incorrect or unintended behavior.
Severity	The degree of impact a defect has on the system, users, data or business.
Priority	The relative urgency with which a defect should be addressed.
Traceability	The ability to establish and follow relationships between related artifacts such as requirements, tests, results and defects.

B. SDLC and STLC

Term	Formal revision definition
SDLC	The overall lifecycle of activities used to specify, design, develop, test, deploy and maintain software.
STLC	The testing-focused activities used to analyze test basis, plan, design, implement, execute and complete testing.
Requirement analysis	Identifying testable requirements, risks, dependencies, ambiguities, data and environment needs.
Test planning	Defining scope, approach, resources, schedule, tools, risks, entry criteria and exit criteria.
Test design	Deriving test conditions, test cases, test data and expected results from the test basis.
Test environment	The hardware, software, data, configuration and connected services needed to execute tests.
Test completion	Summarizing testing, coverage, results, outstanding risks/defects and lessons learned.

C. Testing approaches and execution

Term	Formal revision definition
Black-box testing	Test design based on specified behavior and requirements without requiring knowledge of internal implementation.
White-box testing	Test design based on internal structure or implementation of the software.
Grey-box testing	Testing informed by partial knowledge of internal design or architecture while evaluating behavior.
Static testing	Evaluation of a work product without executing the software.
Dynamic testing	Testing that executes software and evaluates runtime behavior.
Functional testing	Testing that evaluates whether software performs functions according to specified requirements.
Non-functional testing	Testing that evaluates quality characteristics describing how well a system performs its functions.
Positive testing	Testing behavior with valid inputs and conditions.
Negative testing	Testing behavior with invalid inputs or prohibited conditions to check that they are handled as specified.
Confirmation testing / retesting	Testing after a fix to confirm that the original defect has been corrected.
Regression testing	Testing after a change to detect adverse effects on previously working functionality.
Smoke testing	A broad, shallow set of checks to determine whether a build is stable enough for further testing.
Sanity testing	A focused check of a specific change or area; exact usage varies by organization.
Exploratory testing	An approach in which test design, execution and learning occur concurrently and guide subsequent tests.
Ad hoc testing	Informal testing without a detailed predefined set of test cases, relying largely on tester knowledge and experience.
End-to-end testing	Testing a complete user/business workflow across relevant components and systems.
API testing	Testing an API's behavior, responses, data handling, error handling and conformance to its contract.
Maintenance testing	Testing changes to an existing operational system, including fixes, enhancements, migrations and environment changes.

D. Non-functional testing types

Type	Definition
Performance testing	Evaluates responsiveness, throughput, resource use or capacity under specified conditions.
Load testing	Evaluates system behavior under a specified or expected workload.
Stress testing	Evaluates behavior under workload beyond normal/expected conditions or other imposed stress.
Scalability testing	Evaluates the ability to handle increasing workload or resources.
Security testing	Evaluates security properties and resistance to relevant threats.
Usability testing	Evaluates effectiveness, efficiency and satisfaction in a specified context of use.
Compatibility testing	Evaluates behavior across specified platforms, browsers, devices, operating systems or configurations.
Reliability testing	Evaluates the ability to perform without failure for a specified time and conditions.
Accessibility testing	Evaluates whether people with disabilities can use the product against applicable accessibility requirements.
Recovery testing	Evaluates recovery after failures or interruptions.

E. Test design techniques

Technique	Definition
Equivalence partitioning	Divides an input/output domain into partitions expected to be processed similarly; representative values are selected from partitions.
Boundary value analysis	Derives tests from the boundaries of equivalence partitions and nearby values.
Decision table testing	Systematically derives tests from combinations of conditions and their corresponding actions.
State transition testing	Derives tests from states, events, transitions and any relevant guards/actions.
Use case testing	Derives tests from actor-system interactions used to achieve a goal, including main, alternative and exception flows.
Statement testing	A white-box technique that derives tests to execute executable statements.
Branch testing	A white-box technique that derives tests to exercise decision outcomes/branches.
Condition testing	A white-box technique that evaluates individual Boolean conditions as true and false.
Data-flow testing	Derives tests from definitions, uses and changes of data within a program.
Error guessing	An experience-based technique using knowledge of likely defects and common mistakes.
Checklist-based testing	Uses a list of checks or quality items to guide testing.

F. Test levels

Level	Definition
Unit testing	Testing individual units or components, usually in isolation.
Integration testing	Testing interactions between integrated components or systems.
System testing	Testing an integrated system against specified system requirements.
System Integration Testing (SIT)	Testing interfaces and interactions between integrated systems, subsystems or external services; usage varies across organizations.
Acceptance testing	Testing whether a system meets acceptance criteria, business needs or intended-use requirements.
User Acceptance Testing (UAT)	Acceptance testing performed by or with intended users, business representatives or customers to determine suitability for intended use.

G. Agile and Scrum

Term	Definition
Agile	An approach emphasizing collaboration, incremental delivery, feedback and adaptation to change.
Scrum	A lightweight framework for generating value through adaptive solutions to complex problems.
Product Backlog	An ordered, evolving list of work needed to improve the product.
Sprint	A fixed-length event of one month or less during which a usable Increment is created.
Sprint Backlog	The Sprint Goal, selected Product Backlog items and plan for delivering the work.
User story	A common way to express a need from a user's perspective; not a mandatory Scrum artifact.
Epic	A large body of work commonly decomposed into smaller stories; tool and organization usage varies.
Acceptance criteria	Conditions a work item must satisfy to be accepted in its business/product context.
Definition of Done	A formal shared description of the state of the Increment when it meets required quality measures.
Sprint Planning	Event in which the team establishes why the Sprint is valuable, what can be done and how work will be accomplished.
Daily Scrum	A short daily event for Developers to inspect progress toward the Sprint Goal and adapt the plan.
Sprint Review	Event to inspect the Increment and discuss outcomes and future adaptations with stakeholders.
Sprint Retrospective	Event to inspect how the Sprint went and identify improvements in quality and effectiveness.

H. Jira, Xray and Confluence

Tool / concept	Revision definition
Jira	Work and issue-tracking platform for stories, tasks, bugs, workflows, ownership and progress.
Xray	Jira-integrated test-management solution for tests, plans, sets, executions, results and traceability, depending on configuration.
Confluence	Team documentation and knowledge-sharing platform for strategies, requirements, technical documentation and meeting notes.
Test	In Xray, an item representing what is to be tested, often with steps and expected results.
Test Set	A grouping of tests for organization or reuse.
Test Plan	An organization of tests for a planned testing scope/cycle.
Test Execution	A record of running tests and their outcomes for a particular execution.

I. High-value comparisons

Pair	Remember
QA vs QC	QA focuses on process; QC evaluates product quality.
Verification vs validation	Conformance to requirements vs fitness for user needs/intended use.
Scenario vs test case	What to test vs detailed steps/data/expected result for testing it.
Expected vs actual	What should happen vs what was observed.
Severity vs priority	Impact vs urgency.
Retest vs regression	Confirm a specific fix vs detect adverse effects of a change.
Smoke vs sanity	Broad basic build check vs focused check of a specific area/change.
Functional vs non-functional	What the system does vs how well it does it.
Black-box vs white-box	Behavior/specification-based design vs structure/implementation-based design.
Static vs dynamic	No software execution vs software execution.
EP vs BVA	Representative input partitions vs boundary and near-boundary values.
System testing vs SIT	Integrated system vs interfaces/interactions among integrated systems; usage varies.
Acceptance testing vs UAT	Acceptance is the broader category; UAT is a user/business-oriented form of acceptance testing.

J. Quick worked examples

Boundary value analysis: password length 8–20

Length	Expected
7	Reject
8	Accept
9	Accept
19	Accept
20	Accept
21	Reject

Decision table: access to a report

Authenticated?	Permission?	Expected result
Yes	Yes	Allow
Yes	No	Deny
No	Yes	Deny
No	No	Deny

Workflow to remember

```
Requirement → Story / acceptance criteria → Scenarios → Test cases
→ Test data → Execution → Pass/Fail → Defect (if confirmed)
→ Fix → Confirmation test → Regression → Release decision
```

K. Interview self-test

1. Can testing prove that software has no defects?

No. Testing can reveal defects and reduce risk, but cannot prove their complete absence.

2. Is every failed test a defect?

No. Investigate environment, data, setup, requirement changes and reproducibility first.

3. Can a black-box test be performed at unit level?

Yes. Test level and test-design technique are different classifications.

4. Does 100% statement coverage mean all logic is correct?

No. It does not guarantee branch, condition, requirement or behavior coverage.

5. What should a defect report contain?

Title, environment, preconditions, data, reproducible steps, expected/actual result, evidence, severity and priority according to team practice.

6. What happens after a fix?

Confirmation testing checks the fix; regression testing checks for unintended side effects.

7. Why combine EP and BVA?

EP reduces redundant representative values; BVA targets likely edge-related defects.

8. Who usually performs UAT?

Intended users, business representatives or customers, often supported by QA.

9. What does Xray add to Jira?

Test management and traceability for tests, plans, executions and results, depending on setup.

10. Why not automate every UI test?

UI tests can be slower, brittle and costly to maintain; choose automation based on risk, repeatability, stability and value.

Reference note

For exact standardized terminology, check the current ISTQB CTFL syllabus and official glossary at istqb.org and glossary.istqb.org. These notes are paraphrased study definitions, not a verbatim reproduction of an ISTQB glossary or copyrighted textbook. Team conventions may differ, especially for smoke/sanity terminology, defect workflows and SIT naming.

End of Formal Definitions & Quick Revision